
Software Development Models

Terminology

Software Process defines the way to produce software. It includes

- Software life-cycle model

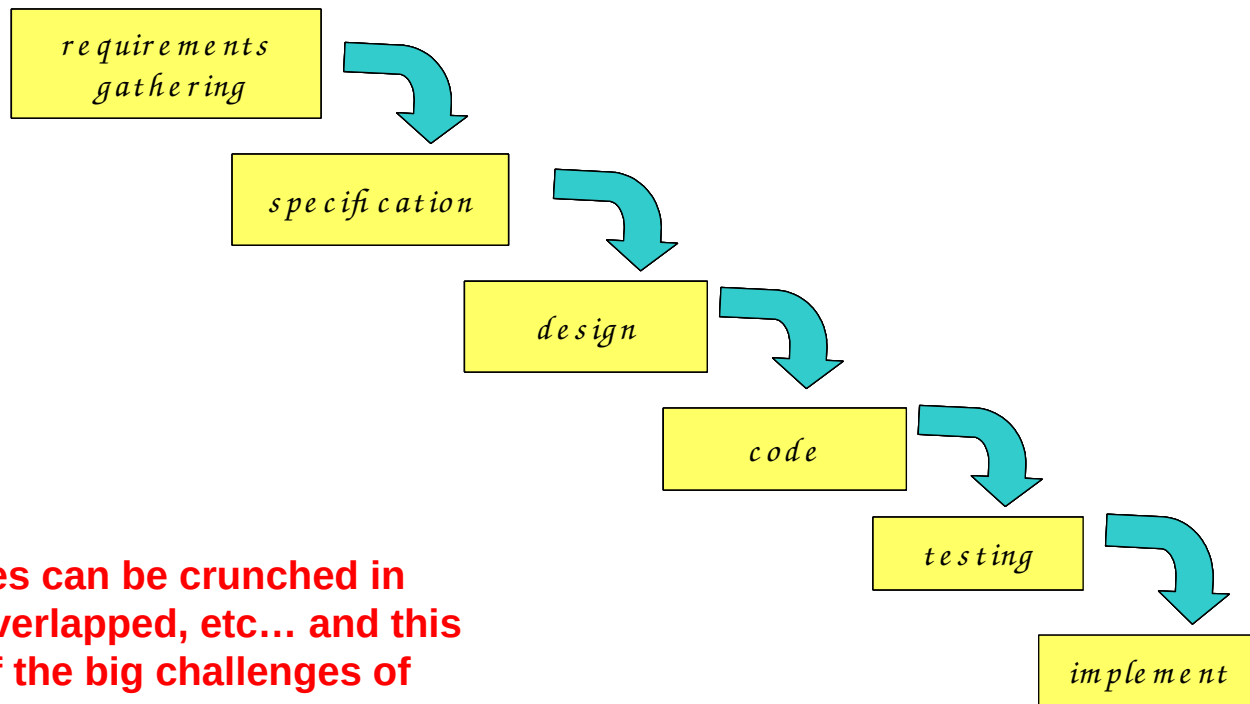
- Tools to use

- Individuals building software

Software life-cycle model defines how different phases of the life cycle are managed.

Software Development Life Cycle (SDLC)

Phases of Software Development



These phases can be crunched in time – overlapped, etc... and this is one of the big challenges of software development in general.

Life-Cycle Models

Build-and-fix model

Waterfall model

Rapid prototyping model

Spiral model

Prototyping Model

Phased Development Model

- incremental development model

- iterative development model

Formal Systems Development

Agile models (model or method)

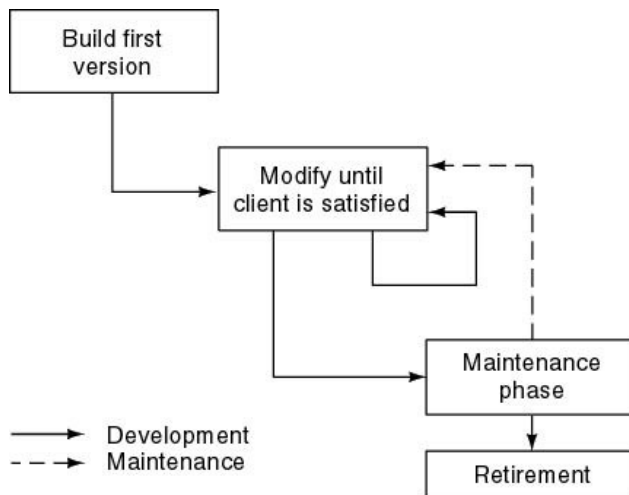
- Extreme programming

- Scrum

- FDD

*See – Ian Sommerville,
Software Engineering, 5 to 9
editions for good chapter on
process models*

Build and Fix Model



Lots of software is developed using build-and-fix model (e.g. student projects)

Basically there is no model.

No specifications

No design

This model is unsatisfactory and should not be adopted.

Why?

Need life-cycle model

“Game plan”

Phases

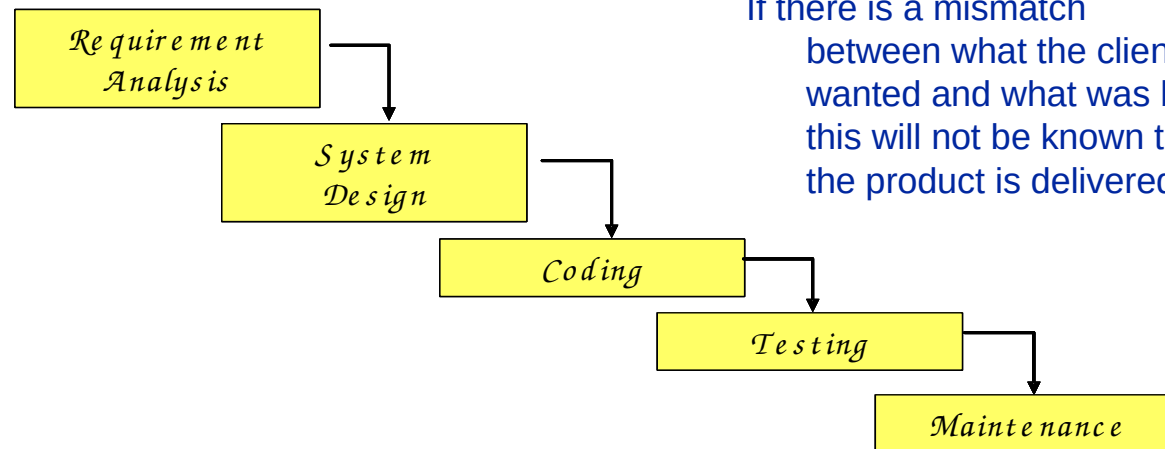
Milestones

Economically untenable

Waterfall Model

Output from one phase is fed as input to the next phase.

One phase is completed, documented and signed-off before the next phase begins.



Advantages

Each phase is well documented.

Maintenance easier? Perhaps.

Disadvantages

If there is a mismatch between what the client wanted and what was built this will not be known till the product is delivered!

Iterative development models

“The process of establishing requirements, designing, building and testing a system, done **as a series of smaller developments**”

Examples are: prototyping, rapid application development (RAD), Rational Unified Process (RUP) and agile development models

Regression testing is increasingly important on all iterations after the first one.

Incremental Model

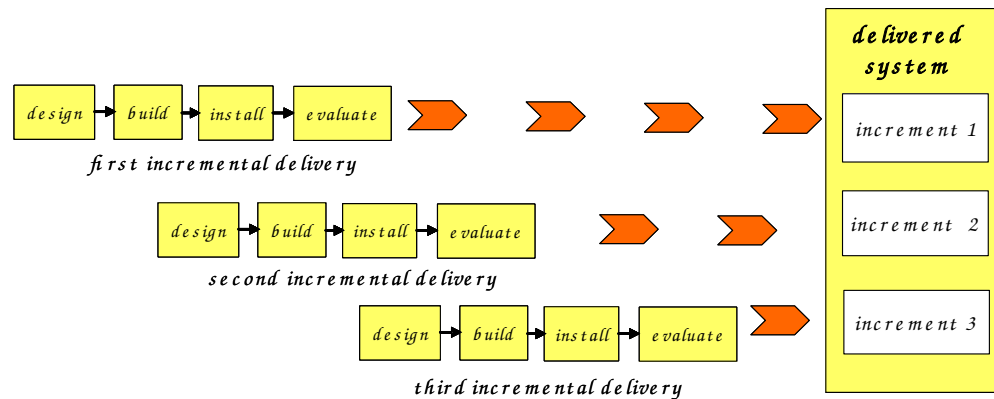
Break system into small components

Construct a partial implementation of a total system

Then **gradually add increased functionality**

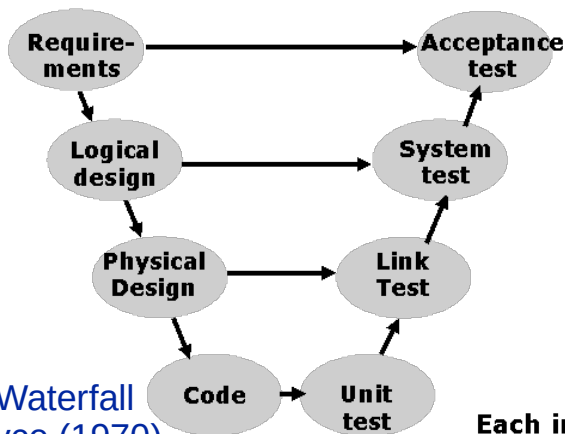
The incremental model **prioritises requirements** of the system and then implements them in groups.

Each subsequent release of the system adds function to the previous release, until all designed functionality has been implemented.



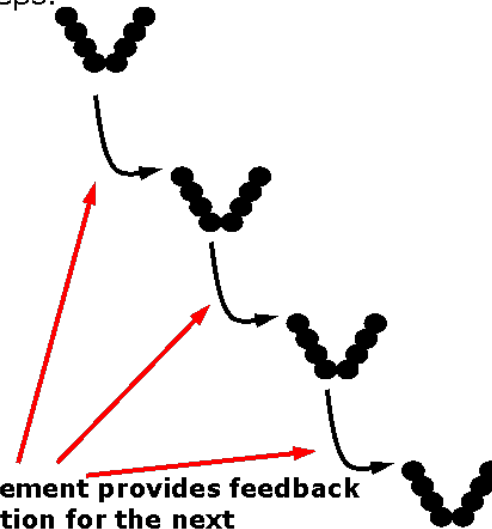
Iterative v Waterfall

Waterfall attempts to solve big problems in one large project.



Note: the creator of the Waterfall model, Winston Royce (1970) actually permitted and encouraged iterative development but this is not generally recognised.

Iterative/Incremental/Evolutionary attempts to solve big problems in a number of smaller steps.



Each increment provides feedback and direction for the next increment

LESS AMBITIOUS => LOWER RISK

Testing within a lifecycle model

Testing within a life cycle model

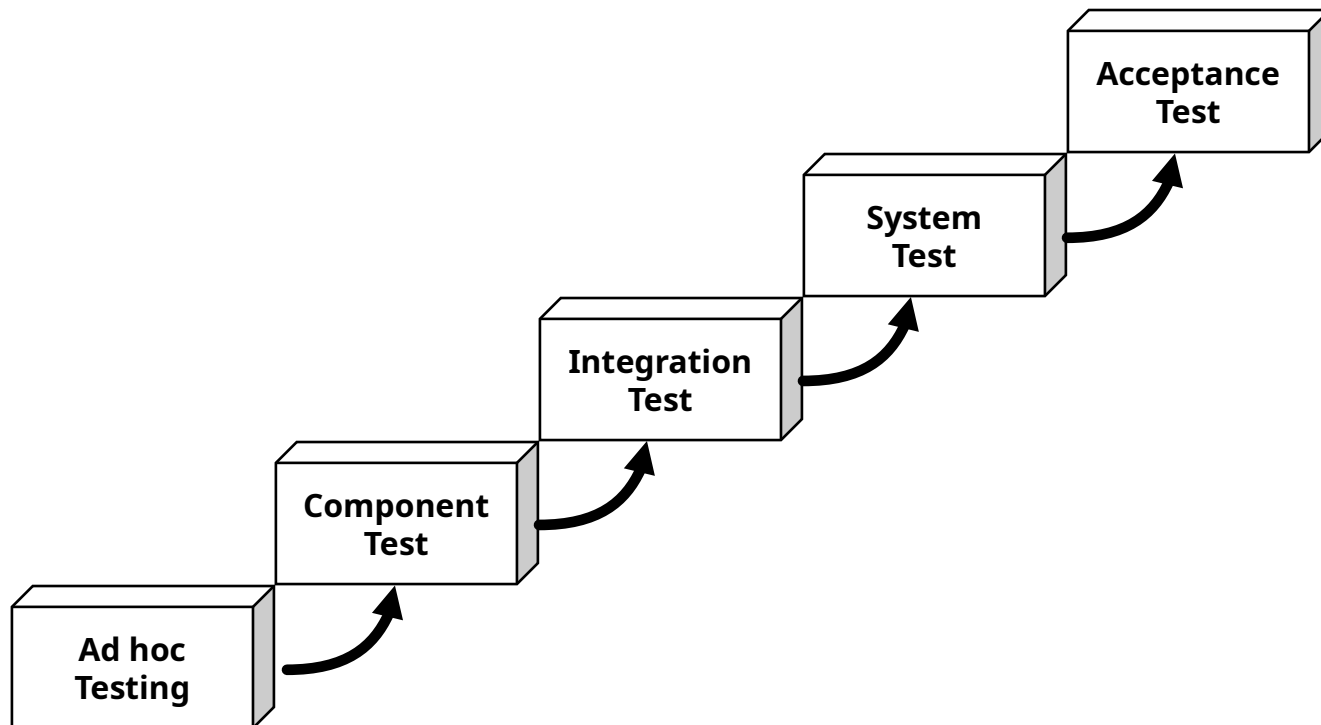
We '**divide and conquer**' the task of testing

Testers should be involved in reviewing documents as soon as drafts are available
in the development life cycle

Testing builds up as we progress through the various stages.

What happens at each stage? How do we determine the objectives for each level?

Typical test process?



From theory to the V-Model

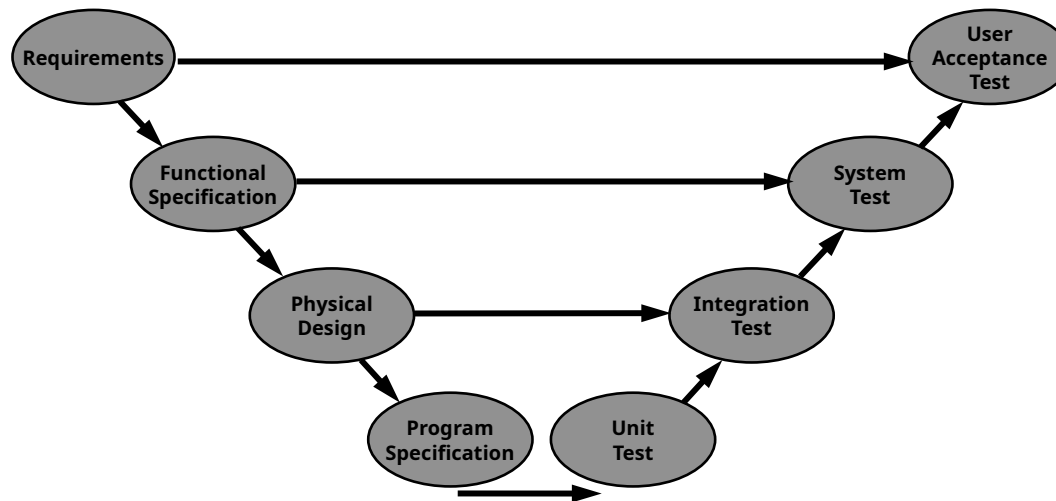
Although variants of the V-model (verification and validation) exist four levels are common

The four commonly levels are:

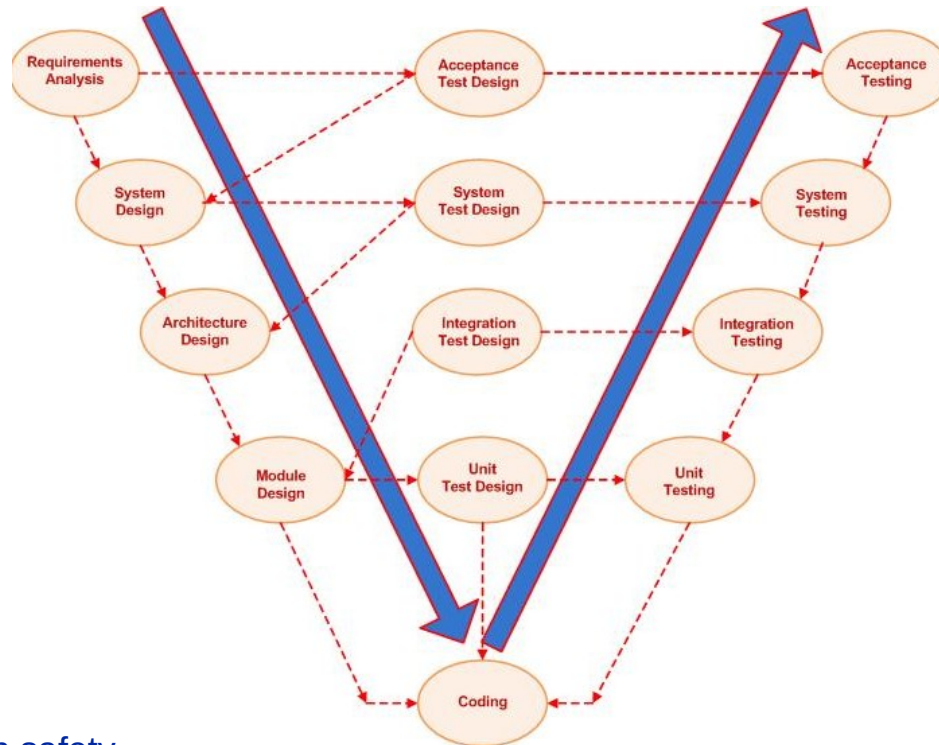
1. component (unit) testing
2. integration testing
3. system testing
4. acceptance testing

V-model may have more, fewer or different levels of development and testing, depending on the project and the software product.

V model: waterfall and locks



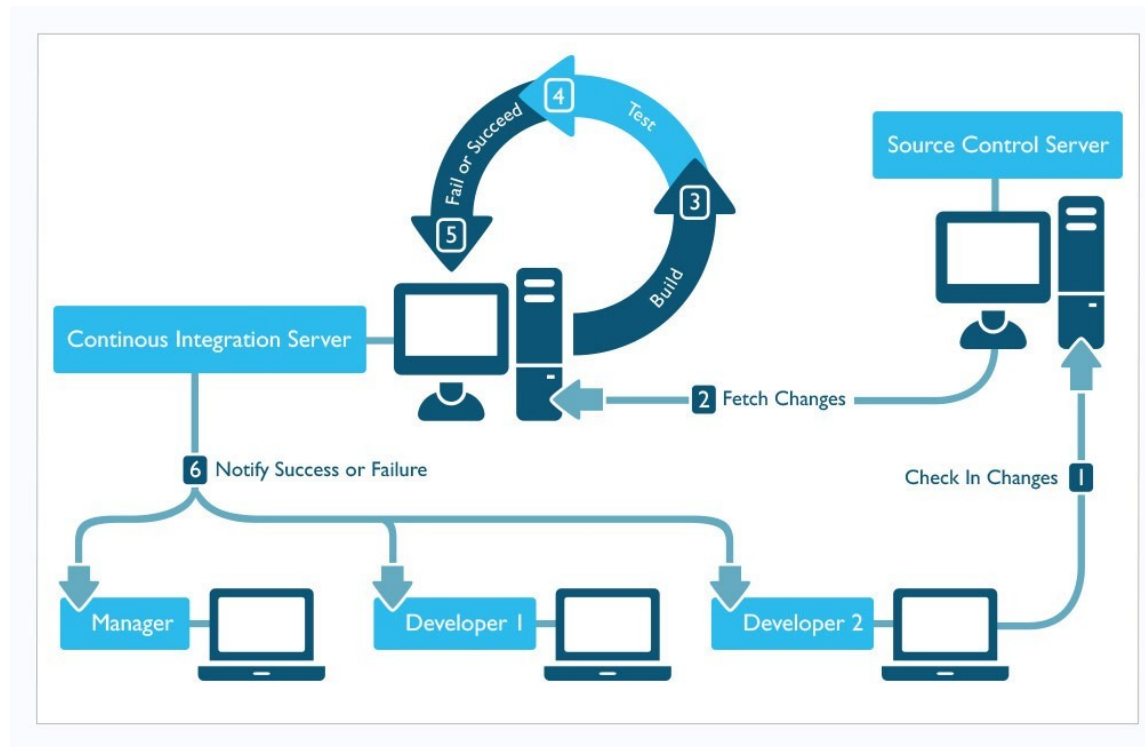
V Model



V-Model is commonly used in safety-critical software development.

Continuous Software Engineering

Continuous Software Engineering (CSE)



Source: <https://insights.sei.cmu.edu/devops/2015/01/continuous-integration-in-devops-1.html>

Continuous Software Engineering - Tooling

Examples of Tooling Used in CSE

Phase	Tool
Development	Fuge - http://fuge.io/
	Seneca - http://senecajs.org/
	HAPI - http://hapijs.com/
Repository	Git / github - https://github.com/
	BitBucket - https://bitbucket.org/
Build	Jenkins - https://jenkins.io/
	Drone - https://drone.io/
Test	Tap - https://testanything.org/
	Phantom - http://phantomjs.org/
	Pdiffy - https://github.com/kennychua/pdiffy
Infrastructure	EC2 - https://aws.amazon.com/ec2/
	ELB - https://aws.amazon.com/elasticloadbalancing/
	Consul - https://www.consul.io/
Deployment	Docker registry- https://docs.docker.com/registry/
	Code Deploy – https://aws.amazon.com/codedeploy/
	Swarm - https://docs.docker.com/swarm/
	Kubernetes - http://kubernetes.io/

Source: O'Connor, R.V., Elger, P., Clarke, P. Continuous Software Engineering – A Microservices Architecture Perspective. To Appear In: Journal of Software Evolution and Process (2017)

Continuous Software Engineering - Continued

Automation/Tooling does not replace existing lifecycles.

Sometimes it is integrated into a traditional lifecycle model – e.g. using Continuous Integration in a V model

Traditional models continue to have an impact but take advantage of tooling where beneficial

Some software engineering has speeded up considerably because of the speed of computer based testing (e.g. DevOps and CSE).

And the tooling is not just related to testing. There is tooling support for many (all?) lifecycle phases.

Influences on the test process

The nature and type of faults to test for

The object under test

Capabilities of developers, testers, users

Availability of: environment, tools, data

The different purpose(s) of testing

- to detect faults

- to demonstrate contractual requirements are met

- to provide evidence that software works and build confidence.

Lifecycle Model Influences Testing

The approach to testing can vary significantly between different lifecycle models

For example, in a traditional V model of software development: requirements are *extensively* elaborated for entire systems up front, these requirements are exhaustively tested prior to coding. The testing emphasis thereafter *may* focus on robust integration and system level testing.

In contrast, in a continuous software engineering approach: software requirements are not exhaustively elaborated for entire systems prior to coding and therefore, the capacity to test the system requirements as a whole up front does not exist. System testing still occurs, but in this scenario a robust focus on unit testing, integration testing and regression testing is emphasized.

Consider the difference between embedded software for a pacemaker (V Model) and a web site that is continually being evolved and extended (CSE). These two scenarios lend themselves to two different software development approaches and consequently, to two different testing strategies.

Not a “black and white” picture?

Aspects of testing that are common in CSE might be useful for traditional V model lifecycles.

Staged testing moves from small to large

We start by testing each component (in isolation)

As tested components become available, we test groups of programs - sub-systems

Then we combine sub-systems and test the system

Then we combine single systems with other systems and test.

Levels of testing have different objectives

The objectives at each level are different

Individual **components are tested for their conformance to their specification**

Groups of components are tested for conformance to the physical design

Sub-systems / **systems are tested for conformance to the functional specifications and requirements.**